

Sorted Linked List ko Balanced Binary Search Tree (BST) mein convert karne ka **Brute Force** tarika samajhte hain.

Brute force ka seedha matlab hai: **Sabse aasan aur bina dimaag lagaye jo dimaag mein aaye**. Linked List ke saath direct khelna mushkil hota hai kyunki hum usmein peeche nahi ja sakte aur random access ($O(1)$) nahi hota. To hum isko ek aasan problem mein badal dete hain.

Yahan pura logic step-by-step diya hai:

1. Brute Force Ki Strategy (The Core Idea)

1. **Linked List ko Array mein badlo**: Pehle poorii Linked List par traverse karo aur uske saare elements ko ek normal Array (ya Vector) mein daal do.
 2. **Array se BST banao**: Kyunki Linked List sorted thi, to hamara Array bhi **Sorted Array** banega. Ab sorted array se BST banana bahut aasan hai (Binary Search ki tarah).
-

2. Step-by-Step Kaise Kaam Karta Hai?

Maaniyo hamare paas yeh Linked List hai:

1 -> 2 -> 3 -> 4 -> 5 -> 6 -> 7

Step 1: Convert to Array

Hum ek loop chalayenge aur elements ko array mein daal denge: `nums = [1, 2, 3, 4, 5, 6, 7]`

Step 2: Find Mid & Make Root

BST ko balanced rakhne ke liye, array ka jo **middle element** hoga, woh hamara **Root** banega.

- Is array ka mid element hai 4.
- To 4 ban gaya hamara Root.

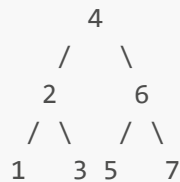
Step 3: Recursion se Left aur Right Subtree banao

- **Left Subtree**: Mid ke left waale elements [1, 2, 3] se banega. Iska mid 2 hai, toh 4 ke left mein 2 aayega.
- **Right Subtree**: Mid ke right waale elements [5, 6, 7] se banega. Iska mid 6 hai, toh 4 ke right mein 6 aayega.

Yeh process recursive tareeqe se chalti rahegi jab tak saare elements tree mein nahi aa jaate.

3. Dry Run (Final Tree Kaisa Dikhega)

Jo array humne liya tha, uska final tree kuch aisa banega:



4. Code Structure (Pseudocode)

Agar iska logic code mein dekhna ho, to woh aisa dikhega:

```

// 1. Helper function jo sorted array ko BST mein badalta hai
Node* sortedArrayToBST(vector<int>& nums, int start, int end) {
    if (start > end) return NULL;

    // Mid element dhoodho
    int mid = start + (end - start) / 2;

    // Mid ko root banao
    Node* root = new Node(nums[mid]);

    // Left aur Right subtree ke liye same kaam karo
    root->left = sortedArrayToBST(nums, start, mid - 1);
    root->right = sortedArrayToBST(nums, mid + 1, end);

    return root;
}

// 2. Main function
Node* sortedListToBST(ListNode* head) {
    vector<int> nums;

    // Linked list ko array mein convert karo
    while (head != NULL) {
        nums.push_back(head->val);
        head = head->next;
    }

    // Array se BST banakar return karo
    return sortedArrayToBST(nums, 0, nums.size() - 1);
}

```

5. Complexity Analysis (Yeh tarika achha hai ya bura?)

- **Time Complexity (Samay):** $O(N)$ List ko array mein badalne mein $O(N)$ laga, aur array se tree banane mein bhi $O(N)$ laga. To total time $O(N)$ hai, jo ki kaafi fast hai!

- **Space Complexity (Jagah):** $O(N)$ Yahan par yeh maat kha jaata hai. Humne ek alag se array banaya saare elements store karne ke liye, isliye $O(N)$ extra space lag gayi.

💡 **Note:** Iska ek **Optimized Tarika (Two Pointer / In-order Traversal)** bhi hota hai, jahan hum bina kisi extra array ke, $O(1)$ extra space mein direct Linked List se BST bana dete hain.

Kya aapko woh optimized tarika bhi samajhna hai, ya is brute force mein koi doubt hai?

Optimized tarika samajhte hain. Brute force mein dikkat kya thi? Hum extra space ($O(N)$) le rahe the array banane ke liye.

Optimized tarike ka goal hai: **Bina koi extra array banaye, direct Linked List ko tree mein convert karna** taaki hamari Extra Space $O(1)$ ho jaye.

Iske do sabshe mashhoor tarike hain. Dono ko detail mein samajhte hain:

Tarika 1: Mid-Element Method (Using Slow & Fast Pointer)

Yeh bilkul waise hi kaam karta hai jaise humne array mein kiya tha: **Middle element dhoodho, use root banao, aur left-right ko recursively solve karo.** Lekin kyunki yeh Linked List hai, mid element dhoodhne ke liye hum **Slow and Fast Pointer (Tortoise and Hare)** algorithm ka use karte hain.

Step-by-Step Logic:

1. **Mid Dhoodho:** Ek **slow** pointer lo aur ek **fast** pointer. **slow** ek kadam chalega, **fast** do kadam. Jab **fast** list ke end mein pahunchega, **slow** bilkul beech (mid) mein hoga.
 2. **List ko Todd do:** Mid se pehle waale node ka connection mid se kaat do, taaki left sub-list alag ho jaye.
 3. **Root Banaao:** **slow** node ke data ko Root bana do.
 4. **Recursion Chalao:** * Head se lekar mid ke pehle tak ki list se **Left Subtree** banega.
- **mid->next** se lekar end tak ki list se **Right Subtree** banega.

Iska Code (C++):

```
ListNode* findMid(ListNode* head) {
    ListNode* prev = NULL;
    ListNode* slow = head;
    ListNode* fast = head;

    while (fast != NULL && fast->next != NULL) {
        prev = slow;
        slow = slow->next;
        fast = fast->next->next;
    }

    // Left part ko break karne ke liye mid ke pehle waale node ka next NULL kar do
    if (prev != NULL) {
        prev->next = NULL;
    }
}
```

```

        return slow; // Yeh hamara mid hai
    }

    TreeNode* sortedListToBST(ListNode* head) {
        if (head == NULL) return NULL;
        if (head->next == NULL) return new TreeNode(head->val); // Ek hi node hai toh
        wahi root hai

        ListNode* mid = findMid(head);

        TreeNode* root = new TreeNode(mid->val);

        // Recursion
        root->left = sortedListToBST(head); // Mid se pehle waali list
        root->right = sortedListToBST(mid->next); // Mid ke baad waali list

        return root;
    }

```

Complexity Analysis:

- **Time Complexity:** $O(N \log N)$ Har baar mid dhoodhne ke liye hume list par travel karna padta hai ($O(N)$), aur tree ki height $\log N$ hoti hai. To total time $O(N \log N)$ ho jata hai.
- **Space Complexity:** $O(\log N)$ Koi extra array nahi banaya, bas recursion ka internal stack space use ho raha hai jo tree ki height ($\log N$) ke barabar hota hai.

Tarika 2: The Ultimate Optimized Method (In-Order Simulation)

Upar waale tarike mein Time Complexity $O(N \log N)$ thi kyunki hum baar-baar mid dhoodh rahe the. Agar interviewer bole ki **Time $O(N)$ chahiye aur Extra Space $O(1)$** , tab yeh sabse best tarika hai.

Iska Asli Logic Kya Hai?

BST ka **In-order Traversal** hamesha ek **Sorted Sequence** deta hai (**Left -> Root -> Right**).

Hamari Linked List bhi pehle se **Sorted** hai.

To hum ulta dimaag lagate hain! Hum Tree ko upar se neeche (Top-Down) banane ke bajaye, **neeche se upar (Bottom-Up)** banayenge, bilkul waise hi jaise In-order traversal hota hai.

Yeh Kaam Kaise Karta Hai?

Hum linked list ke **head** ko ek global ya reference pointer ki tarah treat karenge aur pehle list ki length gin lenge (N).

1. Hum recursion ko bolenge: "*Pehle shuruat ke $N/2$ elements se ek perfect **Left Subtree** bana kar lao.*"
2. Jab recursion left subtree bana kar laayega, hamara **head** pointer apne aap khisak kar list ke **middle element** par pahunch chuka hoga!

3. Ab hum us **head** waale node ko **Root** banayenge.
4. Root banane ke baad, hum **head** ko ek kadam aage badha denge (**head = head->next**).
5. Ab baaki bache elements se **Right Subtree** banayenge.

Iska Code (C++):

```
// List ki total length nikalne ke liye
int countNodes(ListNode* head) {
    int count = 0;
    while (head != NULL) {
        count++;
        head = head->next;
    }
    return count;
}

TreeNode* convertToBST(ListNode* &head, int n) {
    if (n <= 0) return NULL;

    // 1. Pehle Left Subtree banao (n/2 size ka)
    TreeNode* left = convertToBST(head, n / 2);

    // 2. Ab head bilkul sahi jagah par hai, isko Root banao
    TreeNode* root = new TreeNode(head->val);
    root->left = left; // Left ko connect karo

    // 3. Head ko aage badhao agle nodes ke liye
    head = head->next;

    // 4. Ab Right Subtree banao (bache hue elements ka)
    root->right = convertToBST(head, n - n / 2 - 1);

    return root;
}

TreeNode* sortedListToBST(ListNode* head) {
    int n = countNodes(head);
    return convertToBST(head, n);
}
```

Yeh Kyun Sabse Best Hai? (Complexity):

- **Time Complexity:** $O(N)$ Hum har ek node par sirf ek hi baar ja rahe hain. Kahi koi baar-baar loop nahi chal raha.
- **Space Complexity:** $O(\log N)$ Sirf recursion stack ki space lag rahi hai (jo balanced tree mein hamesha $\log N$ hoti hai). Koi extra data structure use nahi kiya.

Ek Line Mein Summary:

- **Brute Force:** List $\rightarrow$ Array $\rightarrow$ BST (Time: $O(N)$, Space: $O(N)$)
- **Optimized 1 (Mid method):** Har baar mid dhoodho (Time: $O(N \log N)$, Space: $O(\log N)$)
- **Optimized 2 (Bottom-Up):** List ke sath-sath In-order tree banao (Time: $O(N)$, Space: $O(\log N)$) - **Best Method!**

Bhai, chill! Bilkul simple shabdon mein, **dry run aur line-by-line code explanation** ke saath samajhte hain.

Hum sabse best tarika (**In-Order Simulation / Bottom-Up Approach**) dekhenge, jo $O(N)$ time leta hai. Yeh interviewer ka favourite hai.

Chhota sa Logic Recall

Hum tree ko upar se nahi, **neeche se (Bottom-Up)** banayenge. Agar hamare paas list hai: **1 -> 2 -> 3** (Length = 3)

1. Pehle recursion left side ke liye chalega aur **1** ko left child banayega.
2. Fir hamara pointer apne aap **2** par aa jayega, usko hum **Root** bana denge.
3. Fir pointer **3** par jayega, usko hum right child bana denge.

Pura Code (C++)

Pehle code dekh lo, fir iska ek-ek karke dry run dekhte hain:

```
#include <iostream>
using namespace std;

// Linked List ka Node structure
struct ListNode {
    int val;
    ListNode* next;
    ListNode(int x) : val(x), next(NULL) {}
};

// Tree ka Node structure
struct TreeNode {
    int val;
    TreeNode* left;
    TreeNode* right;
    TreeNode(int x) : val(x), left(NULL), right(NULL) {}
};

class Solution {
public:
    // 1. Function list ki length (size) nikalne ke liye
    int getLength(ListNode* head) {
        int len = 0;
        while (head != NULL) {
            len++;
            head = head->next;
        }
    }
```

```

        return len;
    }

    // 2. Asli Magic Function (Bottom-Up Helper)
    // Yahan ListNode* &head mein '&' lagana Zaroori hai (Pass by Reference)
    TreeNode* helper(ListNode* &head, int n) {
        // Base Case: Agar nodes khatam ya valid nahi hain
        if (n <= 0 || head == NULL) return NULL;

        // Step A: Pehle Left Subtree banao (Aadhe nodes ka: n/2)
        TreeNode* leftSubtree = helper(head, n / 2);

        // Step B: Ab 'head' pointer sahi bande par khisak chuka hai, isko ROOT
        // banao
        TreeNode* root = new TreeNode(head->val);

        // Jo left subtree pehle banaya tha, use connects karo
        root->left = leftSubtree;

        // Step C: Head ko agle node par badhao (Taaki right subtree ke kaam aaye)
        head = head->next;

        // Step D: Ab bache hue nodes ka Right Subtree banao
        // (Total 'n' tha, 'n/2' left mein gaya, 1 root ban gaya, bacha: n - n/2 - 1)
        root->right = helper(head, n - n / 2 - 1);

        // Root return kar do
        return root;
    }

    // Main Function jo call hoga
    TreeNode* sortedListToBST(ListNode* head) {
        int n = getLength(head); // Pehle length nikal lo
        return helper(head, n); // Fir helper chalao
    }
};

```

&head (Pass by Reference) Ka Khel Kya Hai?

Is code ki sabse badi jaan hai `ListNode* &head`. Agar tum sirf `ListNode* head` likhoge to code fat jayega.

Kyun? Kyunki jab recursion left subtree bana kar aayega, to hum chahte hain ki main `head` pointer agle node par aage badh chuka ho. Agar `&` (reference) nahi lagayenge, to har recursive call apni alag copy banayegi aur `head` pointer wahi ka wahi fasa rahega.

Step-by-Step Dry Run (Sabse Mazedaar Part)

Maano hamari Linked List hai: `10 -> 20 -> 30` (Length $n = 3$) Shuruat mein `head` point kar raha hai `10` par.

****Call 1: helper(head, 3)****

- $n = 3$ hai.
- Yeh line chalegi: `TreeNode* leftSubtree = helper(head, 1);` (Kyunki $3/2 = 1$)
- Yeh abhi ruk gaya aur isne **Call 2** maar di.

****Call 2: helper(head, 1)** (Left subtree ke andar)**

- $n = 1$ hai.
- Yeh line chalegi: `TreeNode* leftSubtree = helper(head, 0);` (Kyunki $1/2 = 0$)
- **Call 3** hui `helper(head, 0)`, jo `NULL` return karegi (Base case).
- Ab **Call 2** wapas aayi. Uska `leftSubtree = NULL` ho gaya.
- Agli line: `TreeNode* root = new TreeNode(head->val);`
- Abhi `head` kiske tha? `10` par. To **Root ban gaya 10**.
- Agli line: `head = head->next;`
- **BOOM!** `head` ab khisak kar `20` par chala gaya.
- Agli line: `root->right = helper(head, 0);` (Kyunki $1 - 0 - 1 = 0$), jo `NULL` dega.
- **Call 2** ne `10` waala node return kar diya **Call 1** ko.

Wapas Call 1 par aate hain:

- **Call 1** ko apna `leftSubtree` mil gaya, jo ki hai node `10`.
- Ab **Call 1** ki agli line chalegi: `TreeNode* root = new TreeNode(head->val);`
- Yaad hai `head` kahan tha? `20` par!
- To **Main Root ban gaya 20**.
- `root->left = leftSubtree;` yani `20` ka left ban gaya `10`.
- Agli line: `head = head->next;`
- **BOOM!** `head` ab khisak kar `30` ke upar chala gaya.
- Agli line: `root->right = helper(head, 1);` (Kyunki $3 - 1 - 1 = 1$)
- Yeh `30` ke liye same Call 2 ki tarah chalega, `30` ka ek node banayega aur `20` ke right mein jod dega.

Final Tree:

```

  20
 /  \

```


10 30

Bina kisi extra array ke, sirf ek hi baar list ghumte-ghumte humne tree bana diya!

Ab batao, abhi bhi koi line ya logic atak raha hai dimaag mein?

Merge Two BSTs (Do Binary Search Trees ko aapas mein jod kar ek single Balanced BST banana) ka **Brute Force** tarika bilkul aasan shabdon mein samajhte hain.

Brute force ka basic funda wahi hai jo humne pehle seekha tha: **Tree ke tough structure ko pehle ek aasan structure (Array) mein badlo, fir solve karo.**

1. Brute Force Ki Strategy (The 3-Step Plan)

Agar hamare paas do alag-alag BST hain, to hum yeh 3 steps follow karenge:

1. **Dono BST ka In-order Traversal nikal kar do alag Arrays banao.**

- Kyunki dono trees BST hain, to unka In-order traversal hamesha **Sorted Array** dega.
- Maan lo, Tree 1 se mila **Array1** aur Tree 2 se mila **Array2**.

2. **Dono Sorted Arrays ko merge karke ek single bada Sorted Array banao.**

- Yeh bilkul **Merge Sort** ke merge step jaisa hai (Two-pointer approach).

3. **Is final Sorted Array se ek naya Balanced BST banao.**

- Yeh wahi step hai jo humne pichli baar seekha tha (Mid element ko root banao aur left-right subtree recursive call se jodo).

2. Ek Mast Example (Dry Run)

Maano hamare paas yeh do BST hain:

BST 1:

```
  3
 / \
1   5
```

BST 2:

```
  4
 / \
```

2 6

Step 1: In-order Traversal (Sorted Arrays)

- Tree 1 ka In-order (Left -> Root -> Right): `arr1 = [1, 3, 5]`
- Tree 2 ka In-order: `arr2 = [2, 4, 6]`

Step 2: Merge Two Sorted Arrays

Hum do pointers use karenge (ek `arr1` par, ek `arr2` par) aur dono ko compare karte hue ek naya merged array banayenge:

- `merged_arr = [1, 2, 3, 4, 5, 6]`

Step 3: Convert Sorted Array to Balanced BST

Ab is `merged_arr` ka mid element dhoodhenge aur tree construct karenge:

- Mid element `3` (ya `4` kuch bhi le sakte ho) root banega.
- Left side se left subtree, right side se right subtree.

3. Pura Code (C++ ke saath)

Ab is pure logic ko code mein utarte hain. Ek-ek function ko dhyan se dekhna:

```
#include <iostream>
#include <vector>
using namespace std;

// BST Node Structure
struct TreeNode {
    int val;
    TreeNode* left;
    TreeNode* right;
    TreeNode(int x) : val(x), left(NULL), right(NULL) {}
};

class Solution {
public:
    // Step 1: BST se sorted array banane ke liye In-order function
    void storeInorder(TreeNode* root, vector<int>& arr) {
        if (root == NULL) return;

        storeInorder(root->left, arr); // Left
        arr.push_back(root->val);      // Root (Store data)
        storeInorder(root->right, arr); // Right
    }
}
```

```
// Step 2: Do sorted arrays ko merge karne ka function (Two-Pointer Technique)
vector<int> mergeSortedArrays(vector<int>& arr1, vector<int>& arr2) {
    vector<int> merged;
    int i = 0, j = 0;

    // Dono arrays mein se chhota element pick karte jao
    while (i < arr1.size() && j < arr2.size()) {
        if (arr1[i] < arr2[j]) {
            merged.push_back(arr1[i]);
            i++;
        } else {
            merged.push_back(arr2[j]);
            j++;
        }
    }

    // Agar arr1 mein kuch bacha ho
    while (i < arr1.size()) {
        merged.push_back(arr1[i]);
        i++;
    }

    // Agar arr2 mein kuch bacha ho
    while (j < arr2.size()) {
        merged.push_back(arr2[j]);
        j++;
    }

    return merged;
}

// Step 3: Sorted Array se Balanced BST banane ka function
TreeNode* sortedArrayToBST(vector<int>& arr, int start, int end) {
    if (start > end) return NULL;

    int mid = start + (end - start) / 2;
    TreeNode* root = new TreeNode(arr[mid]); // Mid bana Root

    root->left = sortedArrayToBST(arr, start, mid - 1); // Left Subtree
    root->right = sortedArrayToBST(arr, mid + 1, end); // Right Subtree

    return root;
}

// MAIN FUNCTION: Jo dono trees ke heads lega
TreeNode* mergeTrees(TreeNode* root1, TreeNode* root2) {
    vector<int> arr1, arr2;

    // 1. Dono trees ka data sorted arrays mein nikalo
    storeInorder(root1, arr1);
    storeInorder(root2, arr2);

    // 2. Dono arrays ko merge karo
    vector<int> mergedArr = mergeSortedArrays(arr1, arr2);
```

```
// 3. Merged array se naya BST bana kar return kar do
return sortedArrayToBST(mergedArr, 0, mergedArr.size() - 1);
}
};
```

4. Complexity Analysis (Interviewer ko batane ke liye)

Maan lo pehle tree mein M nodes hain aur doosre tree mein N nodes hain.

- **Time Complexity:** $O(M + N)$
- Dono trees ka inorder nikalne mein $O(M) + O(N)$ laga.
- Arrays ko merge karne mein $O(M + N)$ laga.
- Naya BST banane mein bhi $O(M + N)$ laga.
- To total time complexiy linear hai, jo ki bahut acchi baat hai!
- **Space Complexity:** $O(M + N)$
- Humne extra arrays (`arr1`, `arr2`, aur `mergedArr`) banaye saare nodes ka data store karne ke liye. Isliye yahan extra space use ho rahi hai.

Summary

Yeh brute force tarika as mein **sabse efficient time complexity** ($O(M + N)$) deta hai, isliye isko interview mein brute force bolne ke baad bhi best time solution mana jata hai. Iska ek aur level-up hota hai jahan hum bina extra array ke, sirf pointers ka use karke trees ko merge karte hain (using Flattening to Doubly Linked List), par interviewers is array waale se kaafi khush ho jaate hain kyunki yeh clean aur error-free hota hai.

Koi step ya code ki line mein confusion hai?

Two Stacks Method (Do Stacks ka use karke do BSTs ko merge karna) ek aur bohot mast tarika hai!

Iska sabshe bada fayda pata hai kya hai? Ismein hume **pura tree ek sath linear structure (array ya DLL) mein nahi badalna padta**. Hum elements ko unke sorted order mein **one-by-one (Iterative In-order)** nikalte hain aur direct unhe compare karte hain.

Chalo isko detail mein aur aasan shabdon mein samajhte hain.

1. Iska Asli Logic Kya Hai?

Jaise hum single BST ka **Iterative In-order Traversal** karne ke liye **ek stack** ka use karte hain, waise hi yahan do trees hain, toh hum **do stacks** (`s1` aur `s2`) ka use karenge.

- `s1` sambhalega **BST 1** ke nodes ko (sorted order mein nikalne ke liye).
- `s2` sambhalega **BST 2** ke nodes ko.

Hum dono stacks se ek-ek karke sabse chhota element top par laayenge, un dono ko compare karenge, jo chhota hoga use pehle nikal kar apne final result (ya array) mein daal denge. Is tarah hume dono trees ka **ek combined sorted data** mil jayega, jisse hum baad mein naya Balanced BST bana lenge.

2. Step-by-Step Algorithm

1. **Leftmost Nodes ko Stack mein Daalo:** BST 1 ke saare leftmost nodes **s1** mein push karo, aur BST 2 ke saare leftmost nodes **s2** mein push karo. (Aisa karne se dono stacks ke **top** par unke respective trees ka sabse chhota element aa jayega).
 2. **Compare Karo:** Dono stacks ke top elements ko dekho:
 - Agar **s1.top()->val < s2.top()->val**: Toh **s1** waale node ko pop karo, uski value ko final list mein daalo, aur ab uske **right child** ke saare leftmost nodes ko **s1** mein daal do.
 - Agar **s2.top()->val <= s1.top()->val**: Toh **s2** waale node ko pop karo, uski value ko final list mein daalo, aur uske **right child** ke saare leftmost nodes ko **s2** mein daal do.
 3. **Loop Chalate Raho:** Yeh tab tak karo jab tak dono stacks khali nahi ho jaate.
-

3. Pura Code (C++ ke saath)

```
#include <iostream>
#include <vector>
#include <stack>
using namespace std;

struct TreeNode {
    int val;
    TreeNode* left;
    TreeNode* right;
    TreeNode(int x) : val(x), left(NULL), right(NULL) {}
};

class Solution {
public:
    // Helper function: Kisi bhi node ke saare left bache-kachhe nodes stack mein
    // daalne ke liye
    void pushLeft(TreeNode* root, stack<TreeNode*>& s) {
        while (root != NULL) {
            s.push(root);
            root = root->left;
        }
    }

    // Step 1 & 2: Do stacks ka use karke sorted order mein elements nikalna
    vector<int> mergeTwoBSTsUsingStack(TreeNode* root1, TreeNode* root2) {
        vector<int> mergedArr;
        stack<TreeNode*> s1, s2;

        // Dono trees ke leftmost elements pehle hi push kar do
```

```

pushLeft(root1, s1);
pushLeft(root2, s2);

// Jab tak dono mein se kisi bhi stack mein maal bacha hai
while (!s1.empty() || !s2.empty()) {

    // Agar stack 2 khali hai, ya fir stack 1 ka element chhota hai
    if (s2.empty() || (!s1.empty() && s1.top()->val < s2.top()->val)) {
        TreeNode* curr = s1.top();
        s1.pop();

        mergedArr.push_back(curr->val); // Chhota element save kiya

        // Uske right child ke left nodes ko push karo
        pushLeft(curr->right, s1);
    }
    // Agar stack 1 khali hai, ya fir stack 2 ka element chhota/barabar
    else {
        TreeNode* curr = s2.top();
        s2.pop();

        mergedArr.push_back(curr->val); // Chhota element save kiya

        // Uske right child ke left nodes ko push karo
        pushLeft(curr->right, s2);
    }
}
return mergedArr;
}

// Step 3: Wahi purana function sorted array se Balanced BST banane ka
TreeNode* sortedArrayToBST(vector<int>& arr, int start, int end) {
    if (start > end) return NULL;
    int mid = start + (end - start) / 2;
    TreeNode* root = new TreeNode(arr[mid]);
    root->left = sortedArrayToBST(arr, start, mid - 1);
    root->right = sortedArrayToBST(arr, mid + 1, end);
    return root;
}

// MAIN FUNCTION
TreeNode* mergeTrees(TreeNode* root1, TreeNode* root2) {
    // 1. Two stack se combined sorted array nikal lo
    vector<int> mergedArr = mergeTwoBSTsUsingStack(root1, root2);

    // 2. Us sorted array se naya tree banao
    return sortedArrayToBST(mergedArr, 0, mergedArr.size() - 1);
}
};

```

4. Ek Chhota sa Dry Run Dekho

Maan lo:

- **BST 1:** Root = 3, Left = 1
- **BST 2:** Root = 4, Left = 2

1. **pushLeft** chalne ke baad:

- **s1** mein neeche se upar hoga: [3, 1] → Top par 1 hai.
- **s2** mein neeche se upar hoga: [4, 2] → Top par 2 hai.

2. Compare hua: 1 < 2.

- **s1** se 1 pop hua, **mergedArr** = [1].
- 1 ka koi right child nahi hai, toh **s1** mein kuch push nahi hua. **s1** mein ab sirf [3] bacha.

3. Agla round compare hua: 3 (s1 top) aur 2 (s2 top). 2 < 3.

- **s2** se 2 pop hua, **mergedArr** = [1, 2].
- 2 ka koi right nahi hai. **s2** mein ab sirf [4] bacha.

4. Agla round: 3 < 4.

- **s1** se 3 pop hua, **mergedArr** = [1, 2, 3]. **s1** ab khali ho gaya.

5. **s1** khali ho gaya, toh loop bache hue **s2** ke 4 ko bhi pop kar dega.

- **mergedArr** = [1, 2, 3, 4].

5. Complexity Analysis

- **Time Complexity:** $O(M + N)$ Kyunki hum dono trees ke har node par maximum 1 ya 2 baar hi ja rahe hain (ek baar push karne aur ek baar pop karne).
- **Space Complexity:** $O(H_1 + H_2)$ ya $O(\log M + \log N)$ **Yahan dhyan dena!** Stacks ke andar ek sath kabhi bhi saare nodes nahi aate. Stack mein sirf tree ki **Height (H)** ke barabar nodes hi aate hain. Isliye stacks ki extra space sirf dono trees ki height ke barabar hogi. (Halanki baad mein array se tree banane ke liye hum array space le rahe hain, par streaming/printing ke liye yeh space $O(H_1 + H_2)$ hi hoti hai).

Conclusion

Yeh approach interviewer tab poochta hai jab woh dekhna chahta hai ki aapko **Iterative Tree Traversal** aata hai ya nahi. Yeh DLL waale tarike se thoda aasan hai kyunki ismein tree ke pointers ko physically todna-jodna nahi padta!

Ab batao, Two-Stack ka maza aaya?

Do BSTs ko merge karne ki **Better/Optimized Approach** samajhte hain.

Upar waale tarike mein dikkat kya thi? Hum elements ko store karne ke liye **vector** (arrays) ka use kar rahe the, jiski wajah se $O(M + N)$ **Extra Space** lag rahi थी.

Optimized approach ka main goal hai: **Extra Space ko kam karke $O(\log M + \log N)$ ya $O(H_1 + H_2)$ par le aana** (jo ki sirf recursion stack ki space hogi), bina koi naya array ya vector banaye.

Optimized Approach Ki Strategy (The Game Plan)

Hum trees ko aapas mein bina extra space ke tabhi jod sakte hain jab unka structure linear ho (jaise Linked List). Isliye hum yeh 3 steps follow karte hain:

1. **BST ko Doubly Linked List (DLL) mein badlo:** Pehle dono BSTs ko in-place (bina extra space) do alag-alag Sorted Doubly Linked Lists mein convert kar lo. (Tree ke **left** pointer ka use **prev** ki tarah aur **right** ka use **next** ki tarah karenge).
 2. **Do Sorted DLLs ko Merge karo:** Jaise hum do sorted linked lists ko merge karte hain, bilkul waise hi dono DLLs ko merge karke ek single badi Sorted DLL bana lo.
 3. **Sorted DLL se wapas Balanced BST banao:** Yeh bilkul wahi tarika hai jo humne sabse pehle seekha tha (Bottom-Up In-order simulation se $O(N)$ mein DLL to BST).
-

Step-by-Step Breakdown with Code

Chalo ab is pure process ko ek-ek karke code ke saath samajhte hain.

Step 1: BST to Sorted DLL (In-Place)

Hum ek helper function likhenge jo Tree ko DLL mein badlega. Ismein **left** child ban jayega **previous** node aur **right** child ban jayega **next** node.

```
// Tree ko Doubly Linked List mein badalne ke liye pointers
TreeNode* prevNode = NULL;
TreeNode* headNode = NULL;

void bstToDLL(TreeNode* root, TreeNode* &head) {
    if (root == NULL) return;

    // A. Pehle Left Subtree par jao
    bstToDLL(root->left, head);

    // B. Current Root ko DLL mein jodo
    if (prevNode == NULL) {
        head = root; // Agar pehla node hai, toh yahi DLL ka head banega
    } else {
        root->left = prevNode;
        prevNode->right = root;
    }
    prevNode = root; // prevNode ko aage badhao

    // C. Ab Right Subtree par jao
    bstToDLL(root->right, head);
}
```


Step 2: Merge Two Sorted DLLs

Ab hamare paas do pointers hain: **head1** (pehle tree ki list) aur **head2** (doosre tree ki list). Inko compare karke ek linear sorted list banani hai.

```
TreeNode* mergeLists(TreeNode* head1, TreeNode* head2) {
    if (!head1) return head2;
    if (!head2) return head1;

    TreeNode* head = NULL;
    TreeNode* tail = NULL;

    // Jo chhota hai usko head banao
    if (head1->val < head2->val) {
        head = tail = head1;
        head1 = head1->right;
    } else {
        head = tail = head2;
        head2 = head2->right;
    }

    // Baaki ke nodes ko jodo
    while (head1 && head2) {
        if (head1->val < head2->val) {
            tail->right = head1;
            head1->left = tail;
            tail = head1;
            head1 = head1->right;
        } else {
            tail->right = head2;
            head2->left = tail;
            tail = head2;
            head2 = head2->right;
        }
    }

    // Agar koi list bach gayi ho
    if (head1) {
        tail->right = head1;
        head1->left = tail;
    }
    if (head2) {
        tail->right = head2;
        head2->left = tail;
    }

    return head;
}
```

Step 3: Merged DLL to Balanced BST ($O(N)$ Method)

Yeh wahi tarika hai jo humne pehle `sortedListToBST` mein seekha tha. Node count ginke Bottom-Up tree banayenge.

```
// Nodes count karne ke liye helper
int countNodes(TreeNode* head) {
    int count = 0;
    while (head) {
        count++;
        head = head->right;
    }
    return count;
}

// Bottom-Up approach se DLL to BST conversion
TreeNode* sortedDLLToBST(TreeNode* &head, int n) {
    if (n <= 0 || head == NULL) return NULL;

    // Left Subtree banao
    TreeNode* left = sortedDLLToBST(head, n / 2);

    // Root banao
    TreeNode* root = head;
    root->left = left;

    // Head ko aage badhao
    head = head->right;

    // Right Subtree banao
    root->right = sortedDLLToBST(head, n - n / 2 - 1);

    return root;
}
```

Sabko Ek Saath Jodte Hain (Main Function)

Ab hamara main function in teeno steps ko ek ke baad ek call karega:

```
TreeNode* mergeTreesOptimized(TreeNode* root1, TreeNode* root2) {
    // Step 1: Dono BSTs ko DLL mein badlo
    TreeNode* head1 = NULL;
    prevNode = NULL; // Global/Class variable reset
    bstToDLL(root1, head1);

    TreeNode* head2 = NULL;
    prevNode = NULL; // Reset fir se
    bstToDLL(root2, head2);

    // Step 2: Dono Sorted DLLs ko merge karke ek single list banao
```

```

TreeNode* mergedHead = mergeLists(head1, head2);

// Step 3: Total nodes gino aur is single list se Balanced BST banao
int totalNodes = countNodes(mergedHead);
return sortedDLLToBST(mergedHead, totalNodes);
}

```

Complexity Analysis (Kyun hai yeh better?)

- **Time Complexity:** $O(M + N)$
- BST to DLL: $O(M) + O(N)$
- Merge Lists: $O(M + N)$
- DLL to BST: $O(M + N)$
- Sab kuch linear time mein ho raha hai, toh total time $O(M + N)$ hi raha.
- **Space Complexity:** $O(H_1 + H_2)$ ya maximum $O(\log M + \log N)$
- Humne koi naya vector, array ya extra nodes create **nahi** kiye. Jo purane tree ke nodes the, unhi ke pointers (**left**, **right**) ko aapas mein manipulate kiya hai.
- Jo thodi bohot space lag rahi hai, woh sirf recursion ke call stack ki hai, jo hamesha balanced tree ke case mein $\log N$ hoti hai.

Summary

Brute force mein humne $O(M + N)$ **extra space** (array) kharch ki thi, jabki is optimized tarike mein humne **In-place memory manipulation** karke space bacha li. Interviewer ko jab tum pehle Brute Force batao aur fir bolo: "Sir, memory optimize karne ke liye main isko Flatten karke DLL mein badal dunga", toh woh full marks de dega!

Bhai, yeh jo problem tum bol rahe ho, isko standard coding mein "**Recover Binary Search Tree**" ya "**Fix BST with Two Nodes Swapped**" kehte hain.

Iska matlab yeh hai ki kisi ne ek acche bhale BST ke **do nodes ki values ko aapas mein badal (swap kar) diya hai**, jisse tree kharab ho gaya hai. Hume bina tree ka structure badle, un dono nodes ko pehchaan kar wapassahi karna hai.

Chalo iska sabse simple **Brute Force** tarika bilkul detail mein samajhte hain code aur dry run ke saath.

1. Brute Force Ki Strategy (The 3-Step Plan)

Iska brute force dimaag lagane ka sabse aasan tarika wahi hai jo hum ab tak seekhte aaye hain—**In-order Traversal**.

Hum jaante hain ki ek sahi BST ka In-order traversal hamesha **Sorted Order** (chhte se bada) hota hai. Agar do nodes swapped hain, toh in-order nikalne par woh array sorted nahi dikhega.

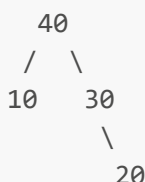
Plan yeh hai:

1. **In-order Traversal karo:** Pura tree ghumo aur saare nodes ki values ko ek vector/array mein store kar lo.
2. **Array ko Sort karo:** Us array ko normal `sort()` function se sahi sorted order mein le aao. (Ab hamare paas ek galat array hai aur ek sahi sorted array hai).
3. **Tree ko Fix karo:** Fir se tree ka in-order traversal karo, aur check karo ki tree ke kis node ki value sahi sorted array se match nahi kar rahi. Jahan mismatch mile, wahan sorted array waali sahi value daal do!

2. Ek Mast Example (Dry Run)

Maano ek sahi BST aisa hona chahiye tha: **10**, **20**, **30**, **40**. Lekin kisi ne **20** aur **40** ko aapas mein badal diya.

Hamara kharab tree kuch aisa dikh raha hai:



Step 1: Store In-order

Jab hum is kharab tree ka In-order (Left -> Root -> Right) nikalenge, toh array milega: `v1 = [10, 40, 30, 20]` (Yeh galat hai, sorted nahi hai)

Step 2: Sort the Array

Humne is array ki ek copy banayi aur use sort kar diya: `v2 = [10, 20, 30, 40]` (Yeh bilkul sahi sorted order hai)

Step 3: Compare aur Fix

Ab hum tree par fir se in-order ghumenge aur `v2` (sahi array) ke saath ek-ek karke match karenge:

- Tree ka pehla in-order node hai **10** → `v2[0]` bhi **10** hai. (Sahi hai, aage badho).
- Tree ka doosra node hai **40** → Lekin `v2[1]` hona chahiye **20**. **Mismatch pakda gaya!** Hum tree ke is node ki value badal kar **20** kar denge.
- Tree ka teesra node hai **30** → `v2[2]` bhi **30** hai. (Sahi hai).
- Tree ka chautha node hai **20** → Lekin `v2[3]` hona chahiye **40**. **Doosra mismatch pakda gaya!** Hum iski value badal kar **40** kar denge.

Tree ho gaya fix!

3. Pura Code (C++)

Ab iska clean code dekh lo:

```
#include <iostream>
#include <vector>
#include <algorithm>
using namespace std;

// BST Node Structure
struct TreeNode {
    int val;
    TreeNode* left;
    TreeNode* right;
    TreeNode(int x) : val(x), left(NULL), right(NULL) {}
};

class Solution {
private:
    // Helper function: Tree se elements ko vector mein daalna
    void storeInorder(TreeNode* root, vector<int>& v) {
        if (root == NULL) return;
        storeInorder(root->left, v);
        v.push_back(root->val);
        storeInorder(root->right, v);
    }

    // Helper function: Tree par fir se ghumna aur values correct karna
    void correctTree(TreeNode* root, vector<int>& sorted_v, int& index) {
        if (root == NULL) return;

        // Left mein jao
        correctTree(root->left, sorted_v, index);

        // Root check karo: Agar value match nahi kar rahi, toh overwrite kar do
        if (root->val != sorted_v[index]) {
            root->val = sorted_v[index];
        }
        index++; // Sahi array ke agle element par jao

        // Right mein jao
        correctTree(root->right, sorted_v, index);
    }

public:
    void recoverTree(TreeNode* root) {
        vector<int> v;

        // 1. In-order nikal kar elements vector mein daalo
        storeInorder(root, v);

        // 2. Ek naya vector banao aur use sort karo (Sahi order ke liye)
        vector<int> sorted_v = v;
```

```

        sort(sorted_v.begin(), sorted_v.end());

        // 3. Tree ko sahi values se fir se fill karo
        int index = 0;
        correctTree(root, sorted_v, index);
    }
};

```

4. Complexity Analysis

- **Time Complexity:** $O(N \log N)$
- Tree ka inorder nikalne mein $O(N)$ laga.
- Array ko sort karne mein $O(N \log N)$ laga.
- Tree ko wapas fix karne mein $O(N)$ laga.
- Toh dominant term hai sorting ki, isliye total time $O(N \log N)$ hai.
- **Space Complexity:** $O(N)$
- Humne do vectors (`v` aur `sorted_v`) banaye nodes ka data store karne ke liye, isliye extra space $O(N)$ lag gayi.

Iska Optimized Tarika Kya Hota Hai? (Short Info)

Interviewer brute force ke baad tumse hamesha kahega: "*Bhai, mujhe $O(N)$ time aur $O(1)$ extra space mein karke dikhao.*"

Usko **Optimized Tarika (Pointer Method)** kehte hain, jahan hum bina koi array banaye, in-order traversal karte-karte hi do pointers (`first`, `second`, aur ek track rakhne ke liye `prev`) ka use karke un dono kharab nodes ko raste mein hi pakad lete hain aur last mein unki values swap kar dete hain.

Kya brute force ka logic sheeshe ki tarah saaf hai, ya iska optimized pointer waala tarika bhi dekhna hai?

Bhai, bilkul sahi pakde ho! Brute force mein hum $O(N \log N)$ time aur $O(N)$ space le rahe the kyunki hum bematlab mein pure array ko sort kar rahe the.

Ab hum bina pure array ko sort kiye, thoda dimaag lagakar ek **Better Approach** sochenge jo $O(N)$ **Time** aur $O(N)$ **Space** mein kaam karegi.

Pehle samajhte hain ki is approach ko **sochna kaise hai (Intuition)**, fir iska code dekhenge.

Socha Kaise? (The Intuition)

Hume pata hai ki BST ka In-order traversal hamesha ek **Sorted Array** hota hai. Maan lo ek sahi array hona chahiye tha: `[10, 20, 30, 40, 50]`

Agar koi bhi do elements swap hote hain, toh do cases bante hain. Un dono cases ko dhyan se dekho:

Case 1: Jab dono kharab nodes door-door ho (Non-adjacent)

Maano 20 aur 50 ko swap kar diya. Array bana: [10, 50, 30, 40, 20] Ab isme gadbadi (mismatch) dhoodho jahan chhota element bade ke baad aa raha ho ($arr[i] > arr[i + 1]$):

- Pehla jhatka laga 50 par, kyunki uske agla element 30 chhota hai. (Yahan bada element 50 hamari **Pehli Galti** hai).
- Doosra jhatka laga 40 par, kyunki uske agla element 20 chhota hai. (Yahan chhota element 20 hamari **Doosri Galti** hai).

Case 2: Jab dono kharab nodes bilkul paas-paas ho (Adjacent)

Maano 20 aur 30 ko swap kar diya. Array bana: [10, 30, 20, 40, 50]

- Yahan sirf ek hi jagah jhatka lagega: 30 par, kyunki agla element 20 chhota hai.
- Is single jhatke mein hi bada element (30) hamari **Pehli Galti** ban jayega aur agla chhota element (20) hamari **Doosri Galti** ban jayega.

Idea: Pure array ko sort karne ki koi zaroorat nahi hai! Bas ek baar In-order array nikal lo, aur usme check karo ki kaun se do elements sorted order ki aisi aisi taisi kar rahe hain. Un dono ko pakdo aur tree mein unki values ko swap kar do.

Step-by-Step Plan (Better Approach)

1. Tree ka normal In-order traversal karke nodes ko ek vector mein store kar lo. (Lekin is baar sirf values nahi, **poore ke poore Node ke pointers** store karenge taaki direct tree mein change kar sakein).
2. Vector par ek single loop chalakar un dono nodes ko dhoodho jo rule tod rahe hain ($arr[i] - > val > arr[i + 1] - > val$).
3. Dono nodes milte hi unki values ko aapas mein `swap()` kar do. Khel khatam!

Pura Code (C++)

```
#include <iostream>
#include <vector>
#include <algorithm>
using namespace std;

struct TreeNode {
    int val;
    TreeNode* left;
    TreeNode* right;
    TreeNode(int x) : val(x), left(NULL), right(NULL) {}
};

class Solution {
private:
```

```

// Helper: Is baar hum values nahi, poore Pointers store kar rahe hain
void storeInorderNodes(TreeNode* root, vector<TreeNode*>& arr) {
    if (root == NULL) return;
    storeInorderNodes(root->left, arr);
    arr.push_back(root); // Pura node daal diya
    storeInorderNodes(root->right, arr);
}

public:
void recoverTree(TreeNode* root) {
    vector<TreeNode*> arr;

    // Step 1: Saare nodes ke pointers in-order mein le aao
    storeInorderNodes(root, arr);

    TreeNode* first = NULL;
    TreeNode* second = NULL;

    // Step 2: Single loop chala kar dono galat nodes ko pehchano
    for (int i = 0; i < arr.size() - 1; i++) {
        // Agar current node agle node se bada hai (Violation!)
        if (arr[i]->val > arr[i+1]->val) {

            // Agar pehli baar aisi gadbadi dikhi hai
            if (first == NULL) {
                first = arr[i];           // Pehla galat node (bada waala)
                second = arr[i+1];       // Temporary doosra node (agar adjacent
hua toh)
            }
            // Agar doosri baar gadbadi dikhi hai (Door waala case)
            else {
                second = arr[i+1];       // Confirm doosra galat node (chhota
waala)
            }
        }
    }

    // Step 3: Dono nodes mil gaye, unki values swap kar do
    if (first != NULL && second != NULL) {
        swap(first->val, second->val);
    }
}
};

```

Complexity Analysis

- **Time Complexity:** $O(N)$
- In-order nikalne mein $O(N)$ laga.
- Loop chalane mein $O(N)$ laga.

- Humne koi sorting nahi ki, isliye total time $O(N \log N)$ se घटकर sirf $O(N)$ ho gaya!
- **Space Complexity:** $O(N)$
- Pointers ko store karne ke liye humne ek vector banaya, isliye space $O(N)$ hai.

Yeh Optimal Kyun Nahi Hai?

Yeh optimal isliye nahi hai kyunki hum abhi bhi $O(N)$ extra space (vector) le rahe hain.

Optimal approach (jisko Morris Traversal ya Pointer-based In-order kehte hain) mein hum yeh vector bhi nahi banate. Hum tree par traverse karte-karte hi live track rakhte hain ki **prev** node kaunsa tha aur **curr** node kaunsa hai, aur real-time mein hi **first** aur **second** ko pakad lete hain. Isse space $O(1)$ ho jaati hai.

Par dimaag chalane ke liye aur interview mein step-by-step jump karne ke liye yeh **Better Approach** sabshe best hai! Ab batao, logic clear hua ki socha kaise?

Bhai, ab aate hain is problem ke **Ultimate Level** yaani **Optimized Approach** par.

Better approach mein humne $O(N)$ time toh achieve kar liya tha, par dikkat wahi thi—hum elements ko store karne ke liye **vector** bana rahe the ($O(N)$ space).

Optimized approach ka target hai: **Bina koi vector/array banaye, real-time mein tree par traverse karte-karte hi dono galat nodes ko pakadna**, taaki extra space $O(1)$ ho jaye.

Socha Kaise? (The Intuition)

Pichli approach mein hum **arr[i]** aur **arr[i+1]** ko compare kar rahe the. Tree traversal mein **arr[i]** ka matlab hota hai **pichla visited node (prev)** aur **arr[i+1]** ka matlab hota hai **current node (curr)**.

Agar hum in-order traversal karte waqt ek pointer rakhlein jo hamesha pichle node ka track rakhe (**prev**), toh hum live check kar sakte hain:

Kya **prev->val > **curr->val** hai?**

Agar yeh condition true hoti hai, toh wahi do cases bante hain jo humne seekhe the:

1. **Pehli baar galti milne par:** **prev** hamara **first** galat node banega, aur **curr** hamara **second** galat node banega.
2. **Doosri baar galti milne par:** **curr** hamara updated **second** galat node banega (door waale case ke liye).

Baas! Jaise hi traversal khatam hoga, hume **first** aur **second** nodes mil chuke honge. Hum bina kisi extra array ke unhe aapas mein swap kar denge.

Step-by-Step Recursion Flow

Hum standard recursive In-order traversal hi karenge, bas teen global/class pointers maintain karenge:

- **prev:** Jo hamesha theek pichle visited node par hoga.
- **first:** Pehla swapped node store karega.

- **second**: Doosra swapped node store karega.

Pura Code (C++)

```
#include <iostream>
#include <algorithm>
using namespace std;

struct TreeNode {
    int val;
    TreeNode* left;
    TreeNode* right;
    TreeNode(int x) : val(x), left(NULL), right(NULL) {}
};

class Solution {
private:
    TreeNode* first = NULL;
    TreeNode* second = NULL;
    TreeNode* prev = NULL;

    // Helper Function: Live In-order Traversal
    void inorder(TreeNode* curr) {
        if (curr == NULL) return;

        // 1. Left Subtree par jao
        inorder(curr->left);

        // 2. ROOT Node Par Kaam (Live Validation)
        if (prev != NULL && prev->val > curr->val) {
            // Agar pehli baar rule Toota hai
            if (first == NULL) {
                first = prev; // Bada element pehli galti hai
                second = curr; // Chhota element temporary doosri galti hai
            }
            // Agar doosri baar rule Toota hai (Nodes door-door hain)
            else {
                second = curr; // Confirm chhota element doosri galti hai
            }
        }

        // Current node ab agle ke liye 'prev' ban jayega
        prev = curr;

        // 3. Right Subtree par jao
        inorder(curr->right);
    }

public:
    void recoverTree(TreeNode* root) {
        // Pointers ko reset karo (Good practice)
```

```

    first = second = prev = NULL;

    // 1. Live traversal chalao aur dono kharab nodes ko dhundho
    inorder(root);

    // 2. Dono nodes mil gaye, unki values swap kar do
    if (first != NULL && second != NULL) {
        swap(first->val, second->val);
    }
}
};

```

Ek Chhota sa Dry Run (Maza Aayega)

Maan lo tree ka in-order sequence banana chahiye tha 1, 2, 3, 4. Kisi ne 4 aur 2 ko swap kar diya, toh sequence bana: 1 -> 4 -> 3 -> 2

1. Shuruat mein `prev = NULL`.
2. Pehla node aaya 1. `prev` abhi NULL hai, toh koi check nahi hoga. `prev` ban gaya 1.
3. Agla node aaya 4. Check hua `prev->val (1) > curr->val (4)`? Nahi. `prev` ban gaya 4.
4. Agla node aaya 3. Check hua `prev->val (4) > curr->val (3)`? **YES! Rule toota.**
 - `first = NULL` tha, toh `first = prev (4)` ban gaya aur `second = curr (3)` ban gaya.
 - `prev` ban gaya 3.
5. Agla node aaya 2. Check hua `prev->val (3) > curr->val (2)`? **YES! Fir se rule toota.**
 - `first` ab NULL nahi hai (woh toh 4 ho chuka hai). Toh `else` block chalega: `second = curr (2)`.
 - `prev` ban gaya 2.
6. Traversal khatam! `first = 4` aur `second = 2` mila.
7. Dono ko swap kar diya → Sequence wapas 1, 2, 3, 4 ho gaya!

Complexity Analysis

- **Time Complexity:** $O(N)$ Hum pure tree par sirf ek hi baar in-order traverse kar rahe hain, isliye time linear ($O(N)$) hai.
- **Space Complexity:** $O(H)$ yaani $O(\log N)$ Humne koi alag se vector ya array nahi banaya. Jo space lag rahi hai, woh sirf recursion ke **Call Stack** ki hai, jo tree ki Height (H) ke barabar hoti hai. Average ya Balanced tree ke liye yeh $O(\log N)$ hoti hai.

Note (For Interviews)

Agar interviewer bohot zyada strict hai aur woh bole ki use **Strict $O(1)$ Auxiliary Space** chahiye (yani recursion stack ki space bhi nahi honi chahiye), tab is code mein recursive in-order ki jagah **Morris Traversal** use kiya jata hai. Morris Traversal tree ke andar temporary links (threads) bana kar bina stack ke traversal karta hai.

Lekin 95% interviews mein yeh pointer-based recursive approach **Perfect Optimized Solution** maani jaati hai kyunki iska code clean hota hai aur isme koi extra data structure use nahi hota.

Bhai, ab batao fix two nodes ke teeno tarike (Brute, Better, aur Optimized) dimaag mein fit baithe ya nahi?

"Largest BST in a Binary Tree" ka **Brute Force** tarika bilkul detail mein samajhte hain.

Pehle problem ka matlab samajho: Tumhe ek normal Binary Tree diya hoga (jo BST nahi hai). Tumhe uske andar ek aisa **Subtree** dhoodhna hai jo apne aap mein ek valid BST ho, aur usme **sabse zyada nodes** honi chahiye (yani uska size largest hona chahiye).

1. Brute Force Ki Strategy (The Top-Down Plan)

Brute force ka sabse seedha dimaag yeh kehta hai: **Tree ke har ek node par jao, check karo ki kya us node se shuru hone wala subtree ek valid BST hai? Agar hai, toh uska size gino aur use update karte raho.**

Hum tree mein upar se neeche (**Top-Down**) challenge:

1. Hum Root node par khade honge aur check karenge: *"Kya yeh pura tree ek valid BST hai?"*
2. Agar pura tree hi BST nikal gaya, toh baat hi khatam! Iska total size hi hamara answer hoga.
3. Agar root wala tree BST **nahi** hai, toh hum uske bachhon par jayenge:
 - Left child par jaakar check karenge ki kya wahan se bada BST mil raha hai.
 - Right child par jaakar check karenge ki kya wahan se bada BST mil raha hai.
4. Dono sides mein se jo bhi **Maximum Size** dega, woh hamara final answer hoga.

2. Iske Liye Kaunse Tools (Functions) Chahiye?

Hume is approach ko code mein badalny ke liye do chote helper functions chahiye honge:

- **isValidBST(root, min, max)**: Yeh batayega ki koi subtree valid BST hai ya nahi.
- **countNodes(root)**: Agar koi subtree BST nikal jata hai, toh yeh uske saare nodes ko ginkar uska size batayega.

3. Pura Code (C++)

Chalo ab is pure logic ko ek clean code mein dekhte hain:

```
#include <iostream>
#include <algorithm>
#include <climits>
using namespace std;

// Binary Tree Node Structure
struct TreeNode {
    int val;
    TreeNode* left;
    TreeNode* right;
};
```

```

    TreeNode(int x) : val(x), left(NULL), right(NULL) {}
};

class Solution {
private:
    // Helper 1: Check karna ki kya koi tree Valid BST hai
    bool isValidBST(TreeNode* root, long long minVal, long long maxVal) {
        if (root == NULL) return true;

        // Agar current node ki value range se bahr hai, toh BST nahi hai
        if (root->val <= minVal || root->val >= maxVal) return false;

        // Left child ke liye max range badlegi, Right child ke liye min range
        badlegi
        return isValidBST(root->left, minVal, root->val) &&
            isValidBST(root->right, root->val, maxVal);
    }

    // Helper 2: Kisi bhi tree ke total nodes (size) count karna
    int countNodes(TreeNode* root) {
        if (root == NULL) return 0;
        // 1 (current node) + left ke nodes + right ke nodes
        return 1 + countNodes(root->left) + countNodes(root->right);
    }

public:
    // MAIN FUNCTION
    int largestBSTSubtree(TreeNode* root) {
        if (root == NULL) return 0;

        // Step 1: Check karo kya current ROOT se shuru hone waala tree BST hai?
        if (isValidBST(root, LLONG_MIN, LLONG_MAX)) {
            // Agar haan, toh iska size hi is subtree ka maximum size hoga
            return countNodes(root);
        }

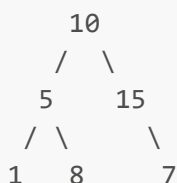
        // Step 2: Agar current root BST nahi hai, toh left aur right subtrees
        mein dhoodho
        int leftLargest = largestBSTSubtree(root->left);
        int rightLargest = largestBSTSubtree(root->right);

        // Dono mein se jo bhi largest size dega use return kar do
        return max(leftLargest, rightLargest);
    }
};

```

4. Ek Mast Dry Run

Maan lo hamara tree kuch aisa dikhta hai:



1. **Root = 10** par check hua: Kya yeh pura tree BST hai?

- Nahi! Kyunki **15** ke right mein **7** hai (jo ki 15 aur 10 dono se chhota hai, rules toot gaye).

2. Ab humne root ke left aur right ko bola ki tum apna apna largest dhoodho:

- **Left side (Root = 5):** Subtree hai **5, 1, 8**. Check hua kya yeh BST hai? Haan! $1 < 5 < 8$ bilkul sahi hai. Iska size gina gaya $\rightarrow 3$.
- **Right side (Root = 15):** Subtree hai **15, 7**. Check hua kya yeh BST hai? Nahi, kyunki **7** chhota hokar right mein hai.

3. Fir **Root = 15** ke bachhon par gaye:

- Uska left NULL hai (0).
- Right mein **7** hai. **7** akela node hai toh valid BST hai $\rightarrow$ Size **1**.

4. Finally, $\max(\text{Left side size}, \text{Right side size})$ yaani $\max(3, 1)$ hua.

5. Answer aaya **3** (jo ki subtree **5, 1, 8** hai).

5. Complexity Analysis (Yeh ganda kyun hai?)

- **Time Complexity:** $O(N^2)$ **worst case mein (Skewed tree mein), Average case:** $O(N \log N)$
- Hum tree ke har node par ja rahe hain (N nodes).
- Aur har node par khade hokar hum fir se pura subtree travel kar rahe hain check karne aur count karne ke liye ($O(N)$).
- Baar-baar same nodes ko visit karne ki wajah se time complexity $O(N^2)$ tak pahunch jaati hai, jo ki bohot bekaar hai.
- **Space Complexity:** $O(H)$
- Humne koi extra data structure nahi banaya, bas recursion stack ki space use ho rahi hai jo tree ki Height (H) ke barabar hai.

Isko Optimize Kaise Kareng? (Intuition)

Interviewer bolega: "Bhai, tum upar se neeche (Top-Down) aa rahe ho, isliye baar-baar same nodes check ho rahe hain. Kuch aisa karo ki kaam $O(N)$ **Time** mein ho jaye."

Optimized Approach (Bottom-Up) ka logic: Hum upar se check karne ke bajaye **neeche (leaves) se shuru karenge**. Har ek node apne parent ko 4 cheezein return karega:

1. Kya woh khud ek valid BST hai? (True/False)
2. Uska total size kitna hai.
3. Us subtree ki Minimum value kya hai.
4. Us subtree ki Maximum value kya hai.

Parent node apne left aur right bachhon se yeh info lekar sirf $O(1)$ time mein check kar lega ki kya woh unke saath milkar ek bada BST bana sakta hai ya nahi. Isse pura tree sirf ek baar traverse hoga aur kaam $O(N)$ mein ho jayega!

Kya brute force ka logic pakad mein aaya? Agar iska woh mast $O(N)$ waala post-order (Bottom-Up) approach chahiye, toh batao, woh bhi detail mein samgha dunga.

Bhai, Largest BST ka **Better/Optimized Approach** samajhte hain. Pichli approach mein dikkat kya thi? Hum upar se neeche (Top-Down) ja rahe the, jiski wajah se ek hi node ko baar-baar valid check karna pad raha tha aur time $O(N^2)$ lag raha tha.

Ab hum dimaag lagayenge aur **Neeche se Upar (Bottom-Up / Post-Order Traversal)** challenge. Isse kaam sirf $O(N)$ **Time** mein ho jayega!

Socha Kaise? (The Intuition)

Maan lo tum ek Parent Node ho. Tumhe kaise pata chalega ki tum apne bachhon ke saath milkar ek valid BST bana rahe ho ya nahi? Tumhe apne Left aur Right bachhon se bas **4 jaankari (Information)** chahiye:

1. **isBST:** Kya left aur right waale subtrees khud mein valid BST hain?
2. **size:** Left aur right subtrees ka size kitna hai?
3. **minVal:** Left subtree ki sabse badi value kya hai? (Kyunki parent ki value left ke har bande se badi honi chahiye).
4. **maxVal:** Right subtree ki sabse chhoti value kya hai? (Kyunki parent ki value right ke har bande se chhoti honi chahiye).

Agar left aur right dono keh dein ki "*Haan, hum BST hain*" aur parent ki value left ke max se badi aur right ke min se chhoti ho, toh parent kehta hai: "*Badhiya! Ab hum sab milkar ek bada BST ban gaye hain.*"

Hum is jaankari ko store karne ke liye ek chhota sa **Structure (Class/Object)** bana lenge jo har node niche se upar pass karega.

Step-by-Step Plan

1. **Post-Order Traversal (Left -> Right -> Root):** Pehle left child par jao, fir right child par jao, fir unse mili info ko root node par process karo.
2. **Base Case:** Agar node **NULL** hai, toh woh ek valid BST hai jiska size 0 hai, uski min value **INT_MAX** hogi aur max value **INT_MIN** (yeh ulta isliye rakhte hain taaki koi bhi valid node isse aasan se compare ho sake).
3. **Validation:** Agar dono bachhe BST hain aur **root->val > left.max** aur **root->val < right.min** hai, toh current root bhi ek bada valid BST ban gaya. Iska size hoga: **1 + left.size + right.size**.
4. **Result Maintain Karo:** Hum ek global variable **maxSize** rakhenge, jise har valid BST milne par update karte rahenge.

Pura Code (C++)

```
#include <iostream>
#include <algorithm>
#include <climits>
using namespace std;

struct TreeNode {
    int val;
    TreeNode* left;
    TreeNode* right;
    TreeNode(int x) : val(x), left(NULL), right(NULL) {}
};

// Ek custom structure jo niche se upar info pass karega
class NodeInfo {
public:
    bool isBST;
    int size;
    int minVal;
    int maxVal;

    NodeInfo(bool isBST, int size, int minVal, int maxVal) {
        this->isBST = isBST;
        this->size = size;
        this->minVal = minVal;
        this->maxVal = maxVal;
    }
};

class Solution {
private:
    int maxBSTSize = 0; // Yeh hamara final answer track karega

    NodeInfo helper(TreeNode* root) {
        // Base Case: Khali node ek valid BST hai
        if (root == NULL) {
            return NodeInfo(true, 0, INT_MAX, INT_MIN);
        }

        // Step 1: Left aur Right subtrees se info mangao (Post-Order)
        NodeInfo left = helper(root->left);
        NodeInfo right = helper(root->right);

        // Step 2: Current node par check karo ki kya yeh BST banta hai?
        if (left.isBST && right.isBST && root->val > left.maxVal && root->val <
            right.minVal) {

            // Haan! Yeh valid BST hai. Iska size nikalo
            int currentSize = 1 + left.size + right.size;
```



```

        // Global maximum size ko update karo
        maxBSTSize = max(maxBSTSize, currentSize);

        // Ab is poore subtree ki min aur max value nikal kar upar parent ko
        bhejo
        int currentMin = min(root->val, left.minVal);
        int currentMax = max(root->val, right.maxVal);

        return NodeInfo(true, currentSize, currentMin, currentMax);
    }

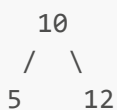
    // Step 3: Agar yeh valid BST nahi bana
    // Toh upar wale parent ko bol do ki hum valid nahi hain (isBST = false)
    // Size ka ab koi matlab nahi toh 0 bhej do, aur min/max ranges bahr kar
    do
        return NodeInfo(false, 0, INT_MIN, INT_MAX);
    }

public:
    int largestBSTSubtree(TreeNode* root) {
        maxBSTSize = 0; // Reset
        helper(root);
        return maxBSTSize;
    }
};

```

Iska Dry Run Samjho

Maan lo ek chhota sa tree hai:



1. **Root = 10** ne bola pehle left (5) par jao.
2. **Node = 5** ke left aur right dono NULL hain. Base case se dono ko mila (`true, 0, INT_MAX, INT_MIN`).
- Node 5 ne check kiya: $5 > \text{INT_MIN}$ (True) aur $5 < \text{INT_MAX}$ (True).
- Node 5 valid BST ban gaya! $\text{Size} = 1 + 0 + 0 = 1$. `maxBSTSize` ab 1 ho gaya.
- Node 5 ne upar bhej diya: (`true, 1, 5, 5`).
3. **Root = 10** ne bola ab right (12) par jao.
- Node 12 bhi same logic se valid BST banega aur upar bhejega: (`true, 1, 12, 12`). `maxBSTSize` abhi bhi 1 hi rahega kyunki size 1 hi mila.

4. Wapas Root = 10 par aaye:

- Left se mila: `left.maxVal = 5`. Right se mila: `right.minVal = 12`.
- Check hua: `10 > 5` (True) aur `10 < 12` (True).
- Aur dono `isBST` bhi true hain.
- **BOOM!** `10` bhi valid ho gaya. $\text{Size} = 1 + 1 + 1 = 3$.
- `maxBSTSize` update hokar ho gaya `3`.

Pure tree mein humne har node par sirf ek hi baar kadam rakha!

Complexity Analysis (Yeh kyun sabshe best hai?)

- **Time Complexity:** $O(N)$ Hum pure tree ka sirf ek baar Post-Order traversal kar rahe hain. Har node par hum sirf $O(1)$ constant time mein math/comparison kar rahe hain. Isliye time complexity linear ($O(N)$) hai.
- **Space Complexity:** $O(H)$ ya $O(\log N)$ Humne alag se koi queue, array ya vector use nahi kiya. Jo space lag rahi hai, woh sirf recursion stack ki hai, jo tree ki height (H) ke barabar hoti hai.

Bhai, yeh approach is problem ki **Sabse Optimized Approach** hai. Ismein humne Top-Down ki galti ko sudhar kar Bottom-Up info pass ki, jisse time $O(N^2)$ se sidha $O(N)$ par aa gaya.

Ab batao, logic poora chamka ya koi doubt bacha hai?

Bhai, yeh jo code humne abhi dekha (Bottom-Up wala), asal mein **wahi iska sabse optimal solution hai!** Isse zyada optimize yeh time ($O(N)$) aur space ($O(H)$) ke maamle mein nahi ho sakta.

Lekin tumhara sawaal bohot gehra hai: "**Bhai, pehle yeh samjha ki kisi insaan ne pehli baar yeh socha kaise hoga?**" (How to build the intuition from scratch?)

Chalo coding ko bhool jaate hain aur ek aam insaan ki tarah sochte hain.

Soch Ka Safar: Step-by-Step

Step 1: Pehli Naakami (Top-Down Soch)

Jab koi naya banda is problem ko dekhta hai, toh uska dimaag hamesha **Top-Down** (upar se neeche) chalta hai. Woh sochega:

- "Main bade waale Root par khada hoon. Pehle isko check karta hoon ki yeh BST hai ya nahi."
- Agar yeh nahi hai, toh main iske left child par jaakar check karunga, fir uske child par...

Dimaag mein jhatka kab laga? Jab usne dekha ki is tarah se ek hi node ko baar-baar check karna pad raha hai. Agar tree bada ho gaya, toh computer hang ho jayega ($O(N^2)$). Yeh dimaag ka dahi karne wala kaam tha. Usne socha, "Main baar-baar neeche jaakar wahi cheez kyun pooch raha hoon? Kya koi aisa tarika hai ki neeche waale khud hi upar aakar mujhe apni report de dein?"

Step 2: Soch Ka Palatna (The Shift to Bottom-Up)

Usne socha, agar mujhe upar khade hokar baar-baar neeche na jaana pade, toh mujhe **neeche (Leaf nodes) se shuru karna chahiye**. Ek single Leaf Node (akela banda jiske koi bachhe nahi hain) hamesha ek **Valid BST** hota hai, jiska size **1** hota hai.

"Wah! Yeh toh bohot aasan hai. Agar main neeche se shuru karoon, toh mujhe shuruat mein hi chhote-chhote valid BST milna shuru ho jayenge!"

Step 3: Pata Kaise Chalega Ki Rishta Kamyab Hai? (The Validation Logic)

Ab usne socha, "Theek hai, leaf nodes ne toh bol diya ki hum valid hain. Lekin unka Parent kaise tay karega ki woh unke saath milkar ek bada BST bana sakta hai?"

Usne BST ka basic rule yaad kiya:

Left waale saare chhote hone chahiye, Right waale saare bade hone chahiye.

Toh Parent ne apne dono bachhon (Left aur Right) se poocha: "Mujhe tumse rishta jodna hai, mujhe kya-kya jaankari chahiye?"

1. **Pehli shart:** Parent ne bola, "Bhai, pehle tum dono khud ek valid BST hone chahiye. Agar tum mein se koi ek bhi badtameez (invalid) nikla, toh main tumhare sath milkar BST nahi bana sakta." (*Yahan se aaya **isBST** ka concept*).
2. **Doosri shart (Left se):** Parent ne Left child se bola, "Tumhare subtree mein bohot saare log hain, mujhe sabse nahi milna. Tum bas apne parivaar ke **sabse bade bande (Maximum Value)** ko mere paas lao. Agar main usse bhi bada hoon, iska matlab main tumhare poore parivaar se bada hoon!" (*Yahan se aaya **maxVal** ka concept*).
3. **Teesri shart (Right se):** Parent ne Right child se bola, "Tum apne parivaar ke **sabse chhote bande (Minimum Value)** ko lao. Agar main usse bhi chhota hoon, iska matlab main tumhare poore parivaar se chhota hoon!" (*Yahan se aaya **minVal** ka concept*).

Step 4: Data ko Batarna (The Information Pack)

Ab usne socha, agar har node apne parent ko yeh saari baatein batana chahta hai, toh woh ek single value (jaise sirf **int**) return karke nahi bata sakta. Kyunki use **True/False** bhi batana hai, **Min** bhi batana hai, aur **Max** bhi batana hai.

"Kyu na main ek **Lifafa (Custom Structure/Class)** bana doon? Har node is lifafe mein apni report bharega aur apne upar waale parent ko pakda dega."

Lifafe mein kya hoga:

- **isBST** (Haan ya Na)
- **size** (Mere parivaar mein kitne log hain)
- **minVal** (Mere parivaar ka sabse chhota)
- **maxVal** (Mere parivaar ka sabse bada)

Step 5: Khel Khatam, Paisa Hazam

Parent ne dono lifafe khole (Left aur Right ke). Agar saari shartein match ho gayi, toh parent ne khushi-khushi apna naya lifafa taiyar kiya:

- Usne apna size nikala: **1 (Main khud) + Left ka size + Right ka size**.
- Usne is size ko apne purane sabse bade size (**maxBSTSize**) se compare kiya, agar yeh bada nikla toh record tod diya!
- Aur apna naya lifafa banakar apne upar waale parent ko bhej diya.

Kahani ka Saar (Conclusion)

Yeh approach isliye sochi gayi kyunki:

1. **Top-Down mein repeat kaam ho raha tha**, toh dimaag ne bola **Bottom-Up** chalo.
2. Bottom-Up chalne ke liye **Post-Order Traversal** (**Left -> Right -> Root**) hi akela rasta hai, kyunki pehle bachhon ka faisla hoga tabhi baap ka faisla hoga.
3. Ek se zyada details pass karni thi, isliye **custom object/structure** dimaag mein aaya.

Bhai, isi ko kehte hain "**Divide and Conquer**" aur "**Trusting the Recursion**". Jab tum badi problem ko aise choti-choti real-life conditions mein todte ho, tab yeh optimal solution nikal kar aata hai.

Ab samajh aaya ki algorithm designer ke dimaag ke ghode kaise daude honge?

Bhai, ab is pure code ke **ek-ek hisse (blocks)** ko chir-faad karke detail mein samajhte hain ki computer ke andar kya chal raha hai.

Pura code teen bado hisson mein bata hua hai. Chalo shuru karte hain:

Hisse 1: Lifafa Taiyar Karna (The **NodeInfo** Class)

Sabse pehle humne ek custom data type (ya class) banaya. Iska kaam sirf itna hai ki yeh recursion mein niche se upar **chaar jaankariyan ek sath** lekar ja sake.

```
class NodeInfo {
public:
    bool isBST;    // Kya yeh subtree ek valid BST hai? (True/False)
    int size;      // Is subtree mein total kitne nodes hain?
    int minVal;    // Is subtree ki sabse CHHOTI value kaunsi hai?
    int maxVal;    // Is subtree ki sabse BADI value kaunsi hai?

    // Constructor: Taaki hum aasani se ek line mein naya lifafa bana sakein
    NodeInfo(bool isBST, int size, int minVal, int maxVal) {
        this->isBST = isBST;
        this->size = size;
        this->minVal = minVal;
        this->maxVal = maxVal;
    }
};
```

Detail: Agar hum yeh class nahi banate, toh functions se ek sath char alag-alag cheezein return karna mushkil hota. Yeh hamara ek packet hai jo har node apne parent ko dega.

Hisse 2: Base Case Ka Khel (Khali Node)

Ab hum apne `helper` function ke andar aate hain, jahan recursion chal raha hai. Sabse pehli line hoti hai Base Case:

```
if (root == NULL) {  
    return NodeInfo(true, 0, INT_MAX, INT_MIN);  
}
```

Yahan ek bohot bada dimaag lagaya gaya hai (Tricky Part):

- Agar koi node `NULL` hai (yani khali hai), toh woh hamesha ek **Valid BST** hota hai (`true`).
- Khali hai toh uska size `0` hoga.
- Lekin humne uski `minVal` ko `INT_MAX` (sabse bada number) aur `maxVal` ko `INT_MIN` (sabse chhota number) kyun rakha?
- **Reason:** Maano ek leaf node hai `5`. Uska left child `NULL` hai. Leaf node `5` check karega ki: "Kya main left child ke `maxVal` se bada hoon?" Yani `5 > left.maxVal`. Agar `left.maxVal` pehle se `INT_MIN` hoga, toh `5 > INT_MIN` hamesha **True** aayega! Isliye ranges ko ulta rakha jata hai taaki parent node hamesha aasani se pass ho jaye.

Hisse 3: Niche se Info Mangana (Post-Order Traversal)

Hum parent node par tabhi faisla le sakte hain jab dono bachhon ki report aa chuki ho. Isliye pehle left aur right par recursive call mari:

```
NodeInfo left = helper(root->left); // Left child ka lifafa aaya  
NodeInfo right = helper(root->right); // Right child ka lifafa aaya
```

Detail: Yeh line tab tak chalegi jab tak recursion bilkul neeche leaf nodes tak nahi pahunch jata. Faisla hamesha sabse neeche se shuru hoga.

Hisse 4: Asli Checking aur Validation (The Logic Gates)

Ab parent node ke paas left aur right dono lifafe hain. Ab woh apni shartein check karega:

```
if (left.isBST && right.isBST && root->val > left.maxVal && root->val <
    right.minVal)
```

Is ek line mein 4 check ho rahe hain:

1. `left.isBST`: Kya left waala parivaar khud mein valid hai?
2. `right.isBST`: Kya right waala parivaar khud mein valid hai?
3. `root->val > left.maxVal`: Kya meri value, left parivaar ke **sabse bade bande** se bhi badi hai?
4. `root->val < right.minVal`: Kya meri value, right parivaar ke **sabse chhote bande** se bhi chhoti hai?

Agar yeh charo baatein **True** hain, toh iska matlab yeh current `root` bhi ek valid BST ka hissa ban gaya!

Agar valid BST ban gaya, toh kya karenge?

```
// 1. Naya size calculate karo
int currentSize = 1 + left.size + right.size;

// 2. Agar yeh size ab tak ka sabse bada hai, toh global answer update karo
maxBSTSize = max(maxBSTSize, currentSize);

// 3. Is poore naye parivaar ka naya min aur max dhoodho upar bhejne ke liye
int currentMin = min(root->val, left.minVal);
int currentMax = max(root->val, right.maxVal);

// 4. Naya lifafa banakar upar bhej do
return NodeInfo(true, currentSize, currentMin, currentMax);
```

Hisse 5: Agar Shart Toot Gayi (Invalid BST)

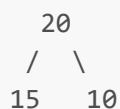
Maan lo upar waali `if` condition fail ho gayi (yani current root par aakar rule toot gaya), tab kya hoga?

```
return NodeInfo(false, 0, INT_MIN, INT_MAX);
```

- Hum upar waale parent ko bol denge `isBST = false` (yani bhai hum narak hain, hamare saath rishta mat jodna).
- Size hum `0` bhej denge kyunki ab iska size aage kisi kaam ka nahi hai.
- `minVal` ko `INT_MIN` aur `maxVal` ko `INT_MAX` bhej dete hain taaki iske upar jo bhi parent ho, woh is gande data ki wajah se khud bhi invalid ho jaye (rule toot jaye).

Pure Flow Ka Ek Animation Jaisa Visualization

Maan lo ek tree hai:



1. 15 valid BST banega aur upar report bhejega: `isBST=true`, `size=1`, `min=15`, `max=15`.
2. 10 valid BST banega aur upar report bhejega: `isBST=true`, `size=1`, `min=10`, `max=10`.
3. Ab main **Root = 20** dono lifafe check karega:
 - Left se mila `maxVal = 15`. Check: `20 > 15`? **Haan (True)**.
 - Right se mila `minVal = 10`. Check: `20 < 10`? **Naa (False!)**.
4. Rule toot gaya! 20 bolega: "Main tum dono ke sath milkar BST nahi bana sakta."
5. 20 upar return kar dega `isBST = false`.
6. Hamara global variable `maxBSTSize` jo pehle 1 ho chuka tha, woh 1 hi rahega. Final answer 1 aa jayega.

Ab batao, code ka ek-ek purza (variable aur if-else) dimaag mein fit baitha ya abhi bhi koi dundhla-pan hai?

Bhai, yeh le poora ka poora code ek sath. Ismein `NodeInfo` class, `helper` function, aur `main` function sab kuch sahi se structured hai.

Isse tu direct apne compiler ya LeetCode par run kar sakta hai:

```

#include <iostream>
#include <algorithm>
#include <climits>

using namespace std;

// 1. Binary Tree Node ka Structure
struct TreeNode {
    int val;
    TreeNode* left;
    TreeNode* right;
    TreeNode(int x) : val(x), left(NULL), right(NULL) {}
};

// 2. Custom Class/Structure jo niche se upar info pass karega (LIFafa)
class NodeInfo {
public:
    bool isBST;    // Kya yeh subtree ek valid BST hai?
    int size;      // Is subtree mein total kitne nodes hain?
    int minVal;    // Is subtree ki sabse CHHOTI value kaunsi hai?
    int maxVal;    // Is subtree ki sabse BADI value kaunsi hai?

    // Constructor: Naya packet (lifafa) banane ke liye
    NodeInfo(bool isBST, int size, int minVal, int maxVal) {
        this->isBST = isBST;
    }
};
  
```

```

        this->size = size;
        this->minVal = minVal;
        this->maxVal = maxVal;
    }
};

class Solution {
private:
    int maxBSTSize = 0; // Global variable jo sabse bade BST ka size track karega

    // 3. Helper Function: Post-Order Traversal (Bottom-Up Approach)
    NodeInfo helper(TreeNode* root) {
        // Base Case: Khali node hamesha ek valid BST hota hai
        if (root == NULL) {
            return NodeInfo(true, 0, INT_MAX, INT_MIN);
            // Trick: minVal=INT_MAX aur maxVal=INT_MIN taaki parent node easily
compare ho sake
        }

        // STEP A: Left aur Right subtrees se report (lifafa) mangao
        NodeInfo left = helper(root->left);
        NodeInfo right = helper(root->right);

        // STEP B: Current 'root' par check karo ki kya yeh valid BST banata hai?
        if (left.isBST && right.isBST && root->val > left.maxVal && root->val <
right.minVal) {

            // Haan! Yeh valid BST hai. Naya size calculate karo
            int currentSize = 1 + left.size + right.size;

            // Agar yeh ab tak ka sabse bada size hai, toh answer ko update karo
            maxBSTSize = max(maxBSTSize, currentSize);

            // Is poore naye subtree ki sabse chhoti (min) aur sabse badi (max)
value nikalo
            int currentMin = min(root->val, left.minVal);
            int currentMax = max(root->val, right.maxVal);

            // Report upar bhej do
            return NodeInfo(true, currentSize, currentMin, currentMax);
        }

        // STEP C: Agar current root par aakar BST ka rule toot gaya
        // Toh upar wale parent ko bol do ki hum valid nahi hain
        return NodeInfo(false, 0, INT_MIN, INT_MAX);
    }

public:
    // 4. MAIN FUNCTION: Jo user ya platform call karega
    int largestBSTSubtree(TreeNode* root) {
        maxBSTSize = 0; // Har naye test case ke liye reset karo
        helper(root); // Bottom-Up process shuru karo
        return maxBSTSize; // Final result return karo
    }
}

```



```
};

// 5. Code ko check karne ke liye ek chhota sa Main Driver Function
int main() {
    /* Hum yeh waala Tree bana rahe hain:
        10
       /  \
      5    15
     / \   \
    1  8   7
    */
    TreeNode* root = new TreeNode(10);
    root->left = new TreeNode(5);
    root->right = new TreeNode(15);
    root->left->left = new TreeNode(1);
    root->left->right = new TreeNode(8);
    root->right->right = new TreeNode(7);

    Solution sol;
    cout << "Largest BST Subtree Size: " << sol.largestBSTSubtree(root) << endl;
    // Output 3 aana chahiye (Subtree 5, 1, 8 wala valid hai)

    return 0;
}
```

Is code ko dry-run karke dekhna, pure tree ke saare nodes par yeh sirf **ek hi baar** visit karega ($O(N)$ Time), isiliye yeh sabse mast aur optimal tarika hai. Aur koi doubt?

"Maximum Sum BST in a Binary Tree" ka **Brute Force** tarika bilkul detail mein samajhte hain.

Yeh problem pichli problem (Largest BST) jaisi hi hai, bas ek chhota sa twist hai: Pichli baar hume woh BST chahiye tha jisme **sabse zyada nodes (size)** hon, is baar hume woh BST chahiye jiske **saare nodes ka sum (jod) sabse zyada** ho.

1. Brute Force Ki Strategy (The Top-Down Plan)

Brute force ka seedha funda: **Tree ke har ek node par jao, check karo ki kya wahan se shuru hone wala subtree ek valid BST hai? Agar hai, toh uske saare nodes ka sum nikalo aur use apne maximum sum se compare karke update karte raho.**

Hum tree mein upar se neeche (**Top-Down**) challenge:

1. Hum main Root node par khade honge aur check karenge: "Kya yeh poora tree ek valid BST hai?"
2. Agar poora tree BST nikal jata hai, toh hum iske saare nodes ka sum nikalenge.
3. Agar root wala tree BST nahi hai, toh hum uske bachhon (**root->left** aur **root->right**) par same cheez recursively call karenge.
4. Poore process mein jahan bhi hume valid BST milega, uske sum ko hum ek global variable **maxSum** mein update karte rahenge.

2. Iske Liye Kaunse Helpers Chahiye?

Hume is approach ke liye teen chhote functions chahiye:

- **isValidBST(root, min, max)**: Yeh check karega ki koi subtree valid BST hai ya nahi.
- **calculateSum(root)**: Agar koi subtree BST nikal gaya, toh yeh uske saare nodes ki values ko plus (+) karke total sum batayega.
- **maxSumBST(root)**: Yeh hamara main function hoga jo top-down traverse karega.

3. Pura Code (C++)

Chalo iska brute force code dekhte hain:

```
#include <iostream>
#include <algorithm>
#include <climits>
using namespace std;

// Binary Tree Node Structure
struct TreeNode {
    int val;
    TreeNode* left;
    TreeNode* right;
    TreeNode(int x) : val(x), left(NULL), right(NULL) {}
};

class Solution {
private:
    int maxSum = 0; // Global variable jo sabse bade sum ko track karega

    // Helper 1: Check karna ki kya koi tree Valid BST hai
    bool isValidBST(TreeNode* root, long long minVal, long long maxVal) {
        if (root == NULL) return true;

        if (root->val <= minVal || root->val >= maxVal) return false;

        return isValidBST(root->left, minVal, root->val) &&
            isValidBST(root->right, root->val, maxVal);
    }

    // Helper 2: Kisi bhi tree ke saare nodes ka Sum nikalna
    int calculateSum(TreeNode* root) {
        if (root == NULL) return 0;
        // Current node ki value + left ka sum + right ka sum
        return root->val + calculateSum(root->left) + calculateSum(root->right);
    }

public:
    // MAIN FUNCTION (Top-Down Brute Force)
    int maxSumBST(TreeNode* root) {
        if (root == NULL) return 0;
```

```
// Step 1: Check karo kya current ROOT se shuru hone wala tree BST hai?
if (isValidBST(root, LLONG_MIN, LLONG_MAX)) {
    // Agar haan, toh iska total sum nikalo
    int currentSum = calculateSum(root);
    // Agar yeh sum ab tak ka sabse bada hai, toh update karo
    maxSum = max(maxSum, currentSum);
}

// Step 2: Left aur Right subtrees par bhi check karo ki kya wahan koi
accha BST hai
maxSumBST(root->left);
maxSumBST(root->right);

// Final maximum sum return kar do
return maxSum;
}
};
```

4. Ek Chhota Sa Dry Run

Maan lo hamara tree kuch aisa dikhta hai (isme negative values bhi ho sakti hain):



1. **Root = 5** par check hua: Kya poora tree BST hai? Nahi, kyunki left mein 9 hai jo 5 se bada hai.
2. Ab recursion left aur right child par gaya:
 - **Left side (Node = 9):** Akela node hai, toh valid BST hai. Sum = 9. **maxSum** update hokar 9 ho gaya.
 - **Right side (Node = 2):** Iska subtree hai 2, 1, 4. Check hua kya yeh BST hai? Haan! $1 < 2 < 4$ bilkul sahi hai.
3. Kyunki 2 wala subtree valid BST hai, toh iska sum nikala gaya: $2 + 1 + 4 = 7$.
4. Humne **maxSum** ko compare kiya: $\max(9, 7)$. Toh **maxSum** 9 hi raha.
5. Final Answer aaya 9 (jo ki akela node 9 de raha hai).

5. Complexity Analysis (Yeh ganda kyun hai?)

- **Time Complexity:** $O(N^2)$ worst case mein
- Hum tree ke har node par ja rahe hain (N nodes).

- Aur har node par khade hokar hum fir se `isValidBST` aur `calculateSum` chala rahe hain jo poore subtree ko travel karta hai ($O(N)$).
 - Baar-baar same nodes ko visit karne ki wajah se time complexity quadratic ($O(N^2)$) ho jaati hai.
 - **Space Complexity:** $O(H)$
 - Sirf recursion stack ki space use ho rahi hai jo tree ki Height (H) ke barabar hai.
-

Isko Optimize Kaise Kareng? (Better/Optimal Approach Ka Intuition)

Interviewer sidha boleگا: "Bhai, mujhe $O(N)$ time mein chahiye!"

Jaise humne **Largest BST** mein kiya tha, bilkul waise hi hum yahan bhi **Bottom-Up (Post-Order Traversal)** challenge.

Har node niche se upar ek lifafa (structure) bhejega jisme yeh information hogi:

1. `isBST`: Kya main valid BST hoon?
2. `sum`: Mere poore subtree ka total sum kitna hai?
3. `minVal`: Mere parivaar ki sabse chhoti value.
4. `maxVal`: Mere parivaar ki sabse badi value.

जब parent node को left और right से ये lifafe मिलेंगे, तो वो $O(1)$ time mein check karega:

- Agar `left.isBST` aur `right.isBST` dono true hain...
- Aur `root->val > left.maxVal` aur `root->val < right.minVal` hai...
- Toh parent boleगा: "Main bhi BST hoon! Mera sum hoga = `root->val + left.sum + right.sum`". Aur hum is sum ko global `maxSum` se compare karke update kar denge.

Isse poora tree sirf ek baar traverse hoga aur time complexity $O(N^2)$ se ghat kar sidha $O(N)$ ho jayegi!

Kya brute force ka logic clear hai? Agar iska optimal ($O(N)$ wala) code bhi chahiye toh batao, woh bhi chhutkiyo mein de dunga!

Bhai, yeh jo LeetCode 1373 (Maximum Sum BST) hai, isko pehli baar kisi insaan ne kaise socha hoga (Intuition), uska poora dimaag ka khel samajhte hain.

Jab hum kisi gande se Binary Tree ko dekhte hain aur usme se sabse bada sum waala BST nikalna hota hai, toh soch 3 steps mein evolve hoti hai:

Step 1: Pehli Soch (The Greedy Trap)

Sabse pehle dimaag bolta hai: "Bhai, sabse upar waale Root ko check karo. Agar poora tree hi BST hai, toh saare nodes ka sum nikal lo, wahi sabse bada hoga."

Lekin phir jhatka lagta hai—agar upar waala root valid nahi hai, toh hume uske left aur right child par jaakar phir se check karna padega (`isValidBST` aur `calculateSum`).

Problem kya aayi? Jab hum upar se neeche (Top-Down) baar-baar ja rahe hain, toh ek hi node ko hazar baar visit kar rahe hain ($O(N^2)$). Dimaag ne bola: "Yeh tarika bekaar hai, computer thak jayega. Mujhe baar-baar

ginti karne neeche nahi jaana, mujhe neeche se upar aana hai!"

Step 2: Soch ka Palatna (Bottom-Up and Post-Order)

Humne socha, agar hum **sabse neeche (Leaf Nodes)** se shuru karein, toh kya hoga?

- Ek akela leaf node (jaise 5) hamesha ek valid BST hota hai.
- Uska sum khud uski value hoti hai (5).
- "Wah! Neeche se toh calculation bohot aasan hai."

Lekin ab sawaal utha: **"Main neeche se upar toh aa raha hoon, par koi bhi baap (Parent Node) apne bachhon ke sath milkar BST bana raha hai ya nahi, yeh kaise decide karega?"**

Yahan lagta hai BST ka asli rule. Parent ko apne bachhon se rishta jodne ke liye 3 cheezein janni thii:

1. **Left aur Right dono se:** "Kya tum dono khud ek valid BST ho? Agar tum dono mein se ek bhi kharab nikla, toh main tumhare sath judkar BST nahi ban sakta."
 2. **Left wale se:** "Tumhare parivaar ka **sabse bada banda (Max Value)** kaun hai? Agar main usse bhi bada hoon, toh main tumhare poore parivaar se bada hoon."
 3. **Right wale se:** "Tumhare parivaar ka **sabse chhota banda (Min Value)** kaun hai? Agar main usse bhi chhota hoon, toh main tumhare poore parivaar se chhota hoon."
-

Step 3: Lifafe ka Idea (The **NodeInfo** Object)

Ab algorithm designer ne socha, agar har node apne upar waale baap ko yeh saari baatein batana chahta hai, toh function se sirf ek **int** (sum) return karke kaam nahi chalega. Kyunki use **True/False (isBST)** bhi batana hai, **Sum** bhi batana hai, **Min** bhi aur **Max** bhi!

"Kyun na main ek **Lifafa (Custom Object/Structure)** bana doon? Har node is lifafe mein apni report bharega aur apne upar waale parent ko pakda dega."

Lifafe ke andar 4 cheezein rakhi gayi: **isBST**, **sum**, **minVal**, **maxVal**.

Step 4: Negative Values ka Dhoka (LeetCode ka Special Twist)

Jab yeh saara logic dimaag mein fit ho gaya, toh ek aakhri dimaag ki batti jali: **"Agar tree mein saare numbers negative (-5, -10, -2) hue toh?"**

Agar hum saare negative numbers ko plus karenge, toh sum aur chhota hota chala jayega (jaise -15). Usse toh accha hai ki hum kisi bhi node ko na lein! Ek **Khali Tree (NULL)** bhi toh valid BST hota hai, aur uska sum 0 hota hai.

Isiliye socha gaya ki hum apne global **maxBSTSum** ko shuruat mein 0 par rakhenge. Agar poore tree mein sirf negative gundey bhare hain, toh hum bolenge: *"Bhai, hume koi node nahi chahiye, humara answer 0 (khali tree) hai."*

Kahani ka Saar (Summary of Thought Process)

1. **Top-Down se thak gaye**, toh dimaag ne bola **Bottom-Up** (Post-Order) chalo.
2. Parent ko check karne ke liye bachhon ki ranges chahiye thi, isliye **Min aur Max ka concept** aaya.
3. Ek sath 4 baatein pass karni thi, isliye **Custom Class (Lifafa)** banayi gayi.
4. Empty tree ka sum 0 hota hai, isliye negative values ko jhelne ke liye global max ko $\emptyset$ se shuru kiya.

Aise dimaag ke ghode dauda kar yeh $O(N)$ ka optimal solution banaya gaya! Ab bilkul andar tak baat samajh aayi ki isko pehli baar kaise socha hoga?

Bhai, Maximum Sum BST ka LeetCode number hai **LeetCode 1373: Maximum Sum BST in Binary Tree**.

Brute force ki $O(N^2)$ ki gandhi complexity ko khatam karne ke liye hum iska **Optimal Solution ($O(N)$ Time)** dekhenge. Logic bilkul wahi hai jo humne Largest BST mein socha tha—**Bottom-Up / Post-Order Traversal** jahan har node niche se upar ek lifafa (structure) bhejega.

Chalo isko पूरी detail aur complete code ke saath samajhte hain.

1. Lifafa Kaise Kaam Karega? (The Custom Structure)

Har node apne parent ko 4 jaankariyan ek sath bhejega:

1. **isBST**: Kya yeh subtree ek valid BST hai? (**true** / **false**)
2. **sum**: Is poore subtree ka total nodes ka sum kitna hai?
3. **minVal**: Is subtree ki sabse chhoti value (Parent ke right side check ke liye).
4. **maxVal**: Is subtree ki sabse badi value (Parent ke left side check ke liye).

2. Base Case Ki Trick (Khali Node)

Agar koi node **NULL** hai:

- Woh valid BST hai $\rightarrow$ **isBST = true**
- Uska sum 0 hoga $\rightarrow$ **sum = 0**
- Aur range ko hum ulta rakhenge $\rightarrow$ **minVal = INT_MAX** aur **maxVal = INT_MIN**. (Taaki leaf node bina kisi dikkat ke apne aap ko valid prove kar sake).

3. Pura Optimal Code (C++)

```
#include <iostream>
#include <algorithm>
#include <climits>
using namespace std;

// LeetCode 1373: Maximum Sum BST in Binary Tree

// Tree Node Structure
struct TreeNode {
    int val;
    TreeNode* left;
    TreeNode* right;
};
```

```

    TreeNode(int x) : val(x), left(NULL), right(NULL) {}
};

// 1. Custom Class jo niche se upar info pass karegi (LIFAFa)
class NodeInfo {
public:
    bool isBST;
    int sum;
    int minVal;
    int maxVal;

    NodeInfo(bool isBST, int sum, int minVal, int maxVal) {
        this->isBST = isBST;
        this->sum = sum;
        this->minVal = minVal;
        this->maxVal = maxVal;
    }
};

class Solution {
private:
    int maxBSTSum = 0; // Global variable jo sabse bade sum ko track karega

    // 2. Helper Function: Post-Order Traversal (Bottom-Up)
    NodeInfo helper(TreeNode* root) {
        // Base Case: Khali node ek valid BST hai jiska sum 0 hai
        if (root == NULL) {
            return NodeInfo(true, 0, INT_MAX, INT_MIN);
        }

        // STEP A: Left aur Right subtrees se report (lifafa) mangao
        NodeInfo left = helper(root->left);
        NodeInfo right = helper(root->right);

        // STEP B: Current 'root' par live check karo ki kya yeh valid BST banata hai?
        if (left.isBST && right.isBST && root->val > left.maxVal && root->val < right.minVal) {

            // Haan! Yeh valid BST hai. Naya combined sum calculate karo
            int currentSum = root->val + left.sum + right.sum;

            // Agar yeh sum ab tak ka sabse bada hai, toh global answer update karo
            maxBSTSum = max(maxBSTSum, currentSum);

            // Is poore naye subtree ki sabse chhoti (min) aur sabse badi (max) value nikalo
            int currentMin = min(root->val, left.minVal);
            int currentMax = max(root->val, right.maxVal);

            // Report upar parent ko bhej do
            return NodeInfo(true, currentSum, currentMin, currentMax);
        }
    }
};

```

```

        // STEP C: Agar current root par rule toot gaya
        // Toh upar wale parent ko bol do ki hum valid nahi hain
        return NodeInfo(false, 0, INT_MIN, INT_MAX);
    }

public:
    // 3. MAIN FUNCTION
    int maxSumBST(TreeNode* root) {
        maxBSTSum = 0; // Reset for every test case
        helper(root); // Bottom-up process shuru
        return maxBSTSum; // Final answer return karo
    }
};

```

4. Code Ka Detailed Post-Mortem (Chir-Faad)

Chalo ek gande case se samajhte hain jahan dimaag ghum sakta hai. Maan lo hamare paas ek node hai `root->val = 5`, uske left se report aayi aur right se report aayi.

- **Left Report:** `isBST = true, sum = 3, minVal = 1, maxVal = 2`
- **Right Report:** `isBST = true, sum = 11, minVal = 6, maxVal = 7`

Computer `if` condition par aayega:

```

if (left.isBST && right.isBST && root->val > left.maxVal && root->val <
    right.minVal)

```

1. `left.isBST (true) && right.isBST (true) → Pass`
2. `root->val > left.maxVal → 5 > 2 → Pass` (5 left ke sabse bade bande se bada hai).
3. `root->val < right.minVal → 5 < 6 → Pass` (5 right ke sabse chhote bande se chhota hai).

Charo conditions **True** ho gayi! Ab code block ke andar jayega:

- `currentSum = 5 + 3 + 11 = 19`
- `maxBSTSum = max(0, 19) = 19` (Answer update ho gaya).
- `currentMin = min(5, 1) = 1`
- `currentMax = max(5, 7) = 7`
- Upar return ho jayega: `NodeInfo(true, 19, 1, 7)`.

5. Complexity Analysis (Interviewer ka Dil Khush)

- **Time Complexity:** $O(N)$ Hum pure tree par sirf **ek baar** visit kar rahe hain (Post-Order). Har node par jo bhi calculation ho rahi hai, woh sirf $O(1)$ constant time le rahi hai (sirf normal `if` checks aur maths). Isliye time complexity quadratic se sidha linear ($O(N)$) ho gayi.

- **Space Complexity:** $O(H)$ ya $O(\log N)$ Humne koi extra array ya vector create nahi kiya. Jo bhi memory use ho rahi hai, woh sirf recursion ke **Call Stack** ki hai, jo tree ki Height (H) ke barabar hoti hai. Worst case (skewed tree) mein yeh $O(N)$ ho sakti hai, par balanced tree mein $O(\log N)$ rahegi.

LeetCode Corner (Important Tip)

Is problem mein Tree ke andar **Negative Values** bhi ho sakti hain (jaise kisi node ki value -5 ho). Lekin hamara global variable `maxBSTSum` shuruat mein 0 par set hai.

Kyun? Kyunki LeetCode ke description ke hisab se, ek **Khali Tree (NULL)** bhi ek valid BST hota hai, aur uska sum 0 hota hai. Agar aapke poore tree mein sirf negative numbers bhare hain (jaise -2 , -3 , -4), toh un sabse accha toh ek khali tree chunna hai jiska sum 0 aaye. Isliye hamara answer kabhi 0 se neeche nahi girega.

Bhai, ab LeetCode 1373 ka poora maza aaya? Teeno blocks (Base case, Post-order call, aur validation) crystal clear hain?